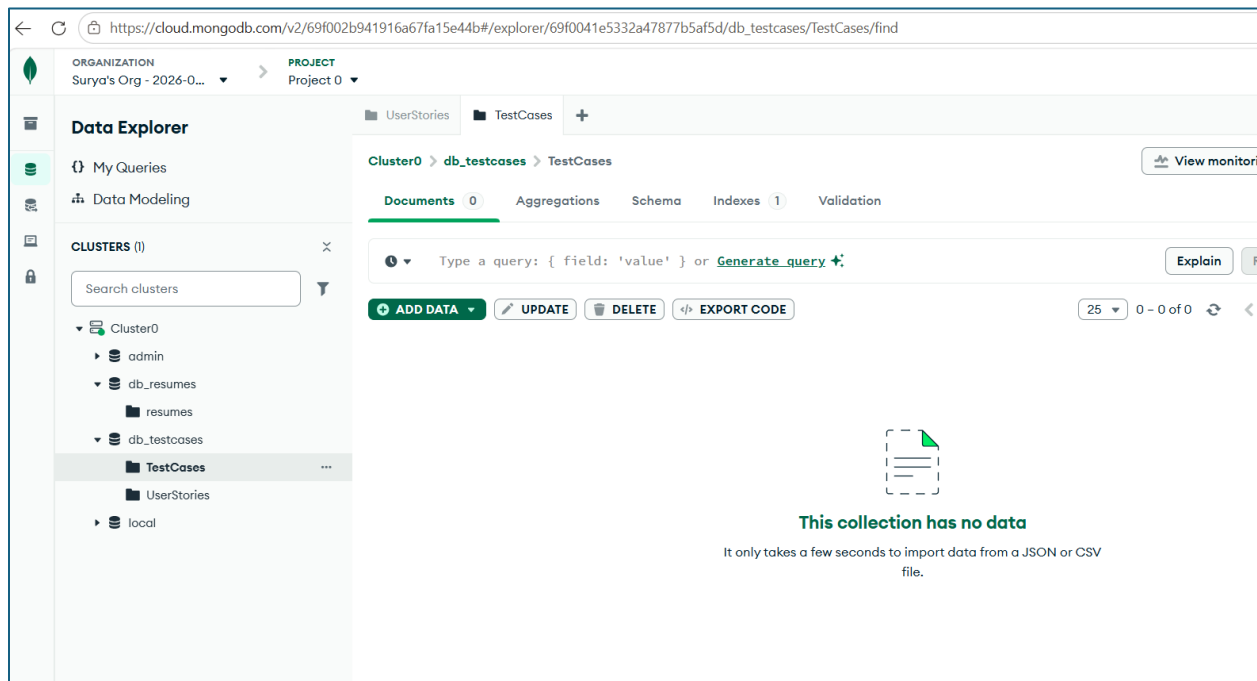


RAG Ingestion - Backend

Step1: Database and Collection created in MongoDB

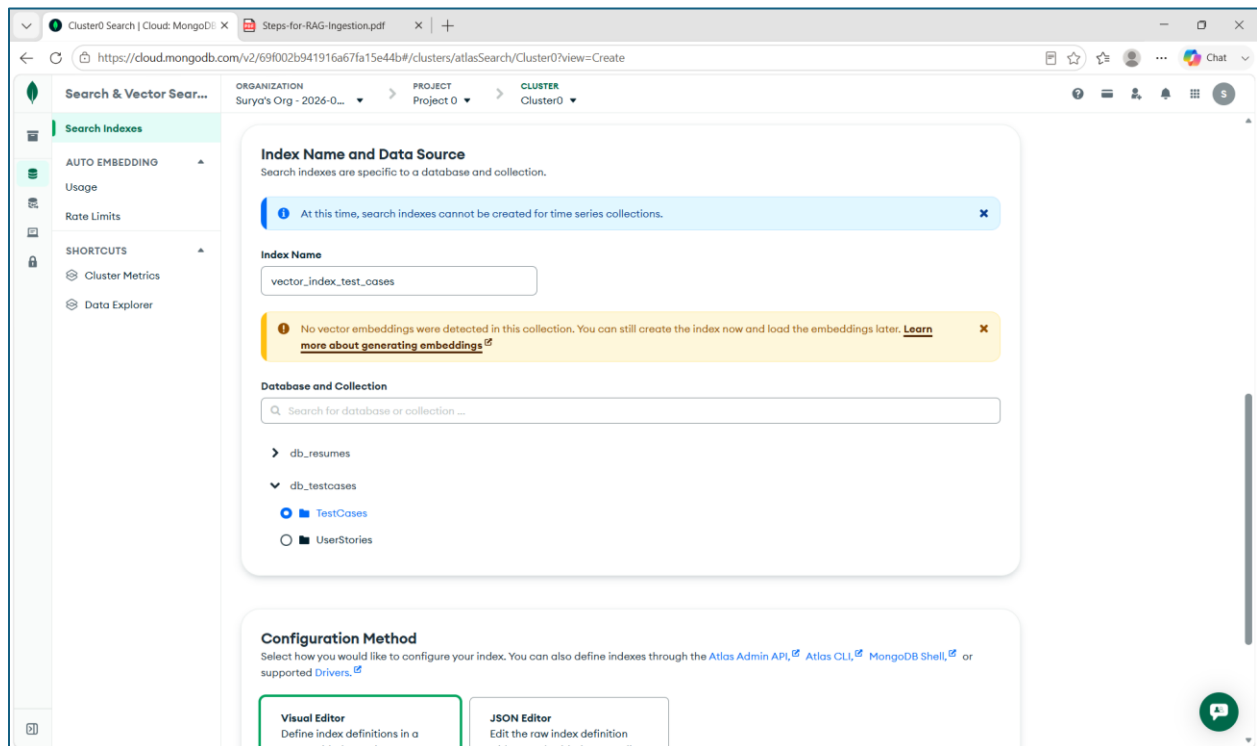
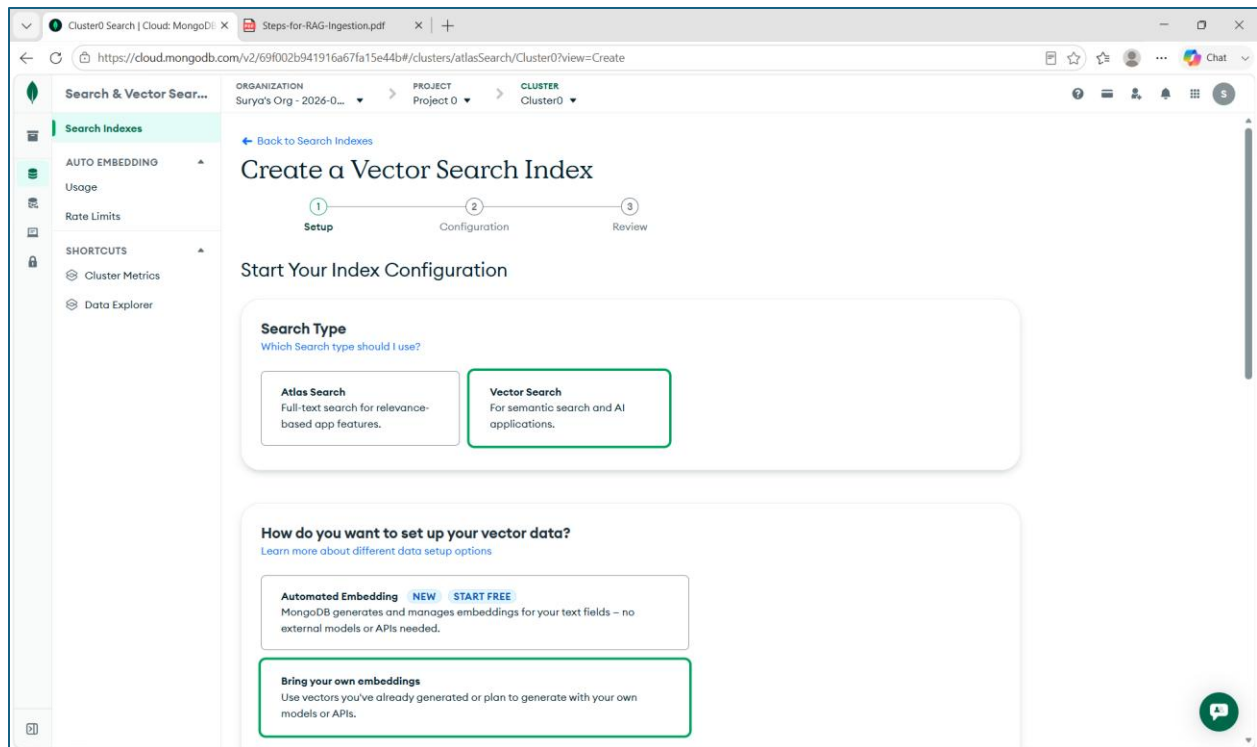
Database: db_testcases

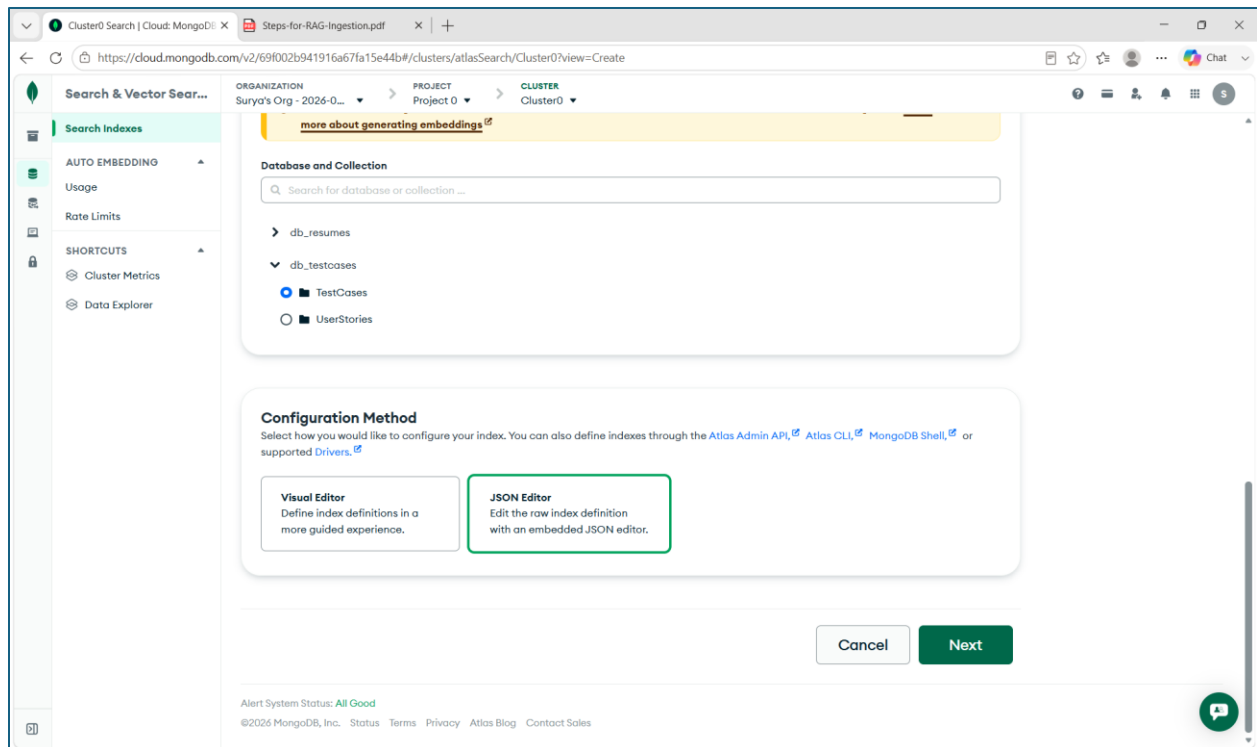
Collection: TestCases



Step2: Creation of Vector Search Index

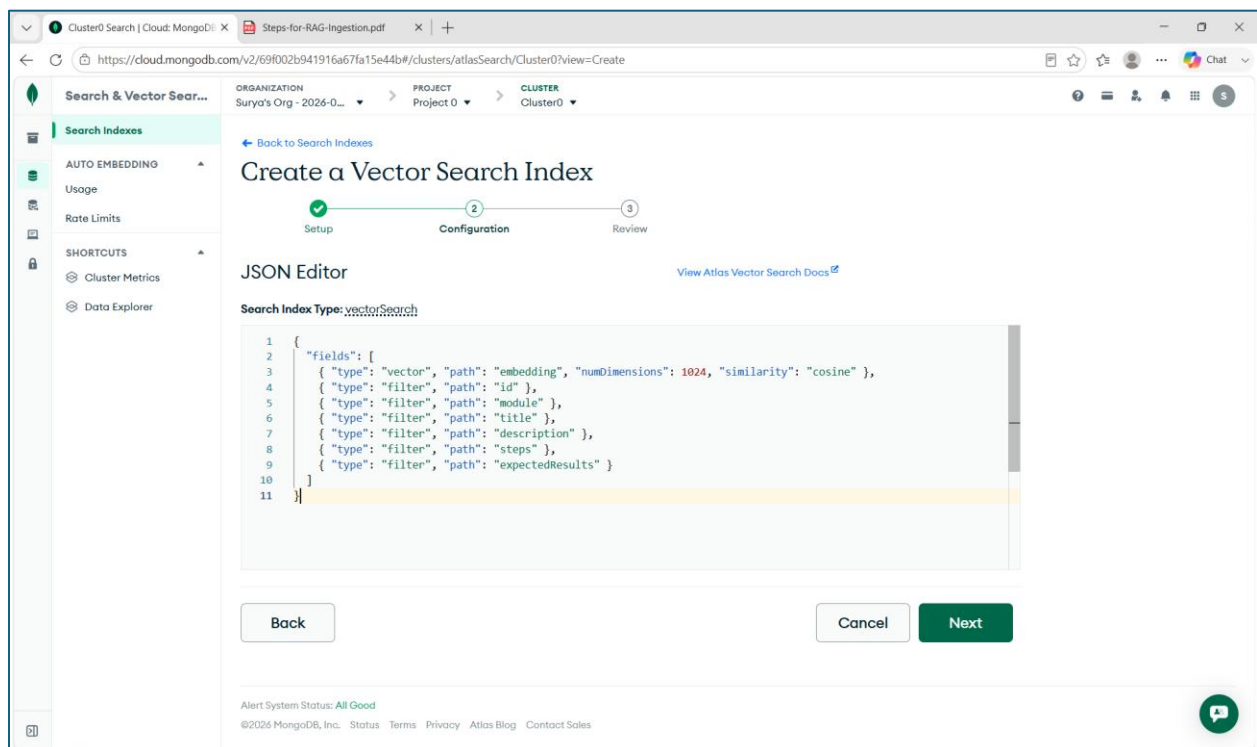
- Select Search Type – Vector Search
- How do you want to set up your data? – select “Bring your own embeddings”
- Give the index name - vector_index_test_cases
- Select the database and collection – db_testcases → TestCases
- Select the configuration method – JSON Editor
- Click Next





Click on Next

Enter the index definition



Click on Next and select “Create Vector Search Index”

The screenshot shows the MongoDB Atlas Search & Vector Search interface. The left sidebar contains navigation links: Search & Vector Search, Search Indexes, AUTO EMBEDDING, Usage, Rate Limits, SHORTCUTS, Cluster Metrics, and Data Explorer. The main panel displays the 'Search & Vector Search' page for 'Cluster0'. A search bar is at the top, and a button 'CREATE SEARCH INDEX' is visible. A blue banner indicates: 'M0 free clusters have a limit of 3 search and vector indexes per cluster. Indexes Used: 1 of 3.' Below this is a table with the following data:

Database	Collection	Index Name	Status	Queryable	Type	Index Fields	Documents (Estimated)	Size
db_testcases	TestCases	vector_index_test_cases	PENDING	✗	vectorSearch	"embedding", "id", "module", "title", "description" ...2 more	0 (0%) indexed of 0	0B

A notification at the bottom left states: 'Index build in progress. Your index vector_index_test_cases is starting to build. View Status Details'.

Step3: The index status in ready state

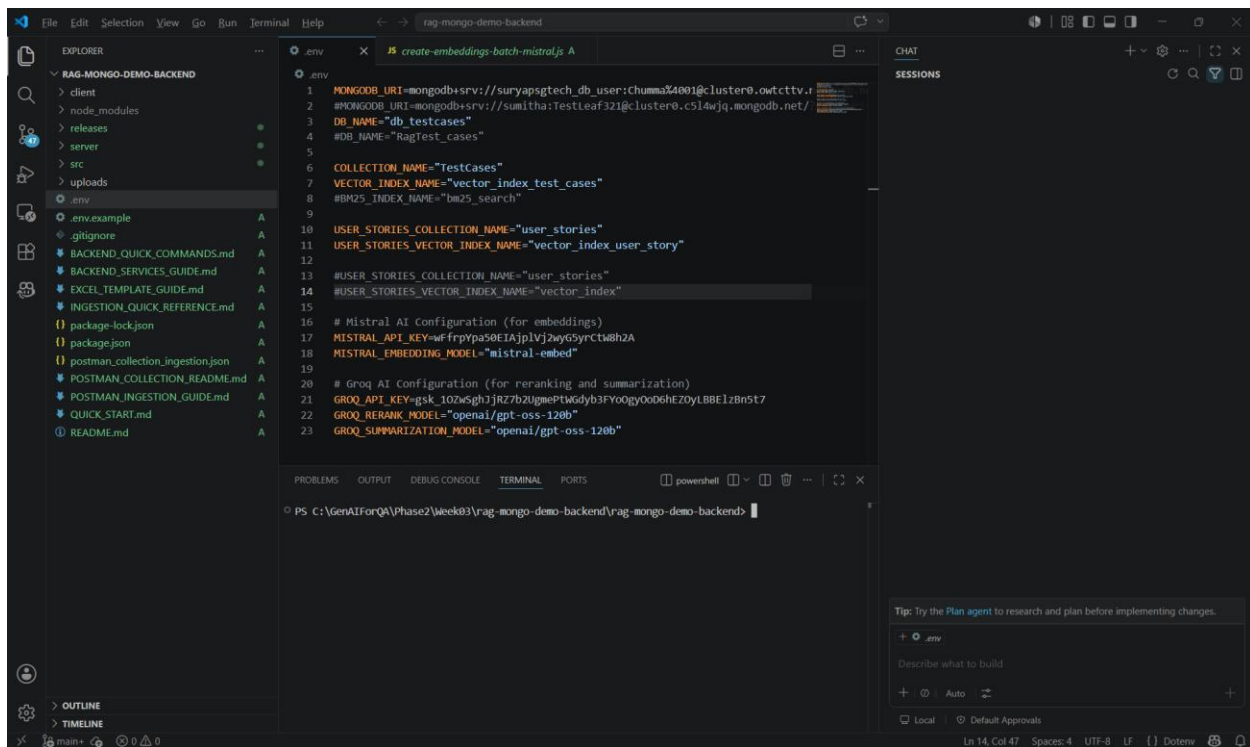
The screenshot shows the MongoDB Atlas Search & Vector Search interface, similar to the previous one, but the index status is now 'READY'. The table data is as follows:

Database	Collection	Index Name	Status	Queryable	Type	Index Fields	Documents (Estimated)	Size
db_testcases	TestCases	vector_index_test_cases	READY	✓	vectorSearch	"embedding", "id", "module", "title", "description" ...2 more	0 (0%) indexed of 0	0B

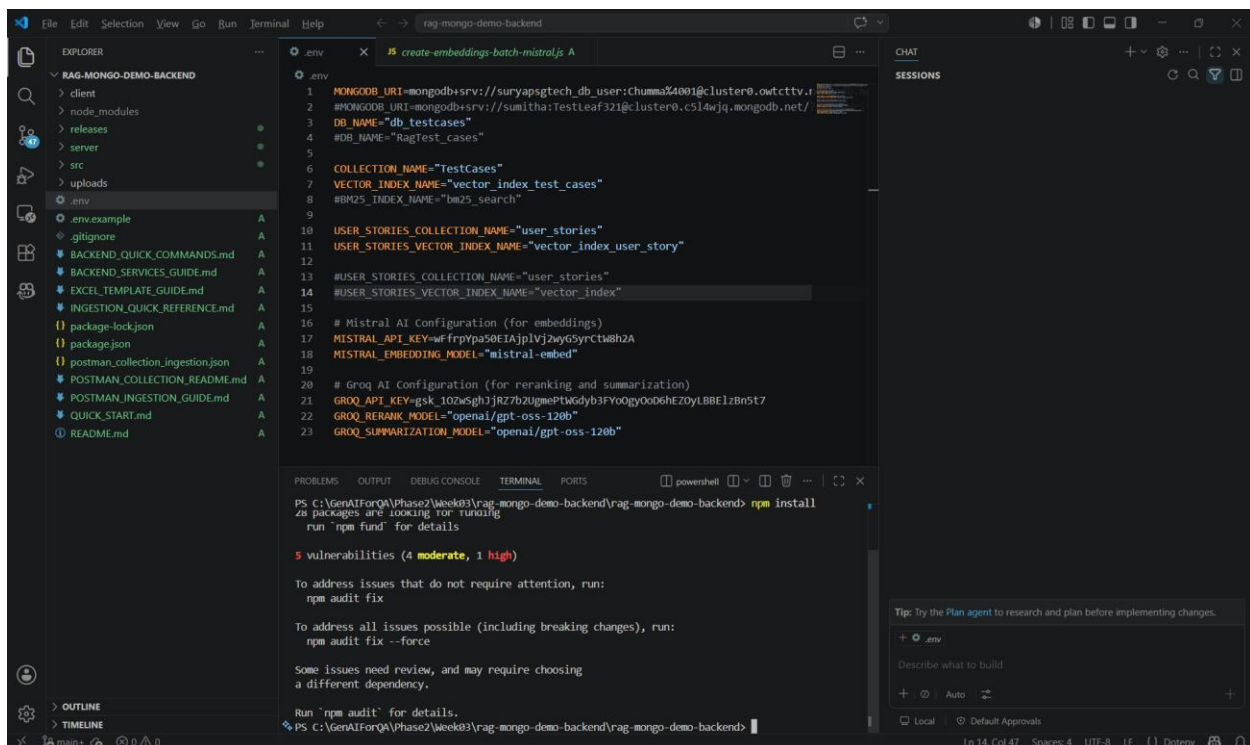
The notification at the bottom left is no longer present. The footer shows 'Alert System Status: All Good' and '©2026 MongoDB, Inc.'.

Step4: Copy the Atlas connection string from Cluster Connection and update the username and password

Step5: Modified .env file



Step6: Run the npm install command to install dependencies



Step7: Run the below scripts

a. node src/scripts/data-conversion/excel-to-json.js

The screenshot shows a VS Code editor with the file explorer on the left, the source code in the center, and the terminal at the bottom. The file explorer shows the project structure with folders like client, node_modules, releases, server, src, config, data, uploads, and env. The source code in the center is a JavaScript file named testcases.json, which contains an array of test case objects. The terminal at the bottom shows the command `node src/scripts/data-conversion/excel-to-json.js` being executed, and the output shows the conversion of 634 rows from 'testcases' into 'src/data/testcases.json'.

```
1 {
2   "module": "Registration",
3   "id": "TC_005",
4   "prerequisites": "PRN/ERN Number",
5   "title": "Conversion from PRN/ERN Reg to UHID",
6   "description": "Verify Conversion from PRN/ERN Reg to UHID",
7   "steps": "1.Launch the application\r\n2.Login to the application with",
8   "expectedResults": "Should get convert successfully from PRN/ERN Reg t",
9   "automationManual": "Automated",
10  "priority": "P1",
11  "createdBy": "Prabakaran",
12  "createdDate": 45165,
13  "lastModifiedDate": 45176,
14  "risk": "Low",
15  "version": "v7.0",
16  "type": "Integration",
17  "linkedStories": "HC-080"
18 },
19 {
20   "module": "Registration",
21   "id": "TC_020",
22   "prerequisites": "Valid UHID",
23   "title": "Sticker Generation",
24   "description": "Verify To Generate Sticker",
25   "steps": "1.Launch the application\r\n2.Login to the application with",
26   "expectedResults": "Stickers should get generate with the number of cc",
27   ...
28 }
```

PS C:\GenAIForQA\Phase2\Week03\rag-mongo-demo-backend\rag-mongo-demo-backend> npm install
To address all issues possible (including breaking changes), run:
npm audit fix --force
Some issues need review, and may require choosing a different dependency.
Run 'npm audit' for details.
PS C:\GenAIForQA\Phase2\Week03\rag-mongo-demo-backend\rag-mongo-demo-backend> node src/scripts/data-conversion/excel-to-json.js
Converted 634 rows from 'testcases' into 'src/data/testcases.json'
PS C:\GenAIForQA\Phase2\Week03\rag-mongo-demo-backend\rag-mongo-demo-backend>

b. Creation of embeddings - node src/scripts/embeddings/create embeddings-batch-mistral.js

The screenshot shows a VS Code editor with the file explorer on the left, the source code in the center, and the terminal at the bottom. The file explorer shows the project structure with folders like client, node_modules, releases, server, src, config, data, uploads, and env. The source code in the center is a JavaScript file named create-embeddings-batch-mistral.js, which contains the configuration for the Mistral AI embedding API. The terminal at the bottom shows the command `node src/scripts/embeddings/create-embeddings-batch-mistral.js` being executed, and the output shows the configuration details and the progress of the embedding creation process.

```
1 MONGODB_URI=mongodb+srv://suryapsstech_db_user:Chumma34001@cluster0.owtcttv.mongodb.net/
2 #MONGODB_URI=mongodb+srv://sumitha:Testleaf321@cluster0.c514wjq.mongodb.net/
3 DB_NAME="db_testcases"
4 #DB_NAME="RagTest_cases"
5
6 COLLECTION_NAME="TestCases"
7 VECTOR_INDEX_NAME="vector_index_test_cases"
8 #BM25_INDEX_NAME="bm25_search"
9
10 USER_STORIES_COLLECTION_NAME="user_stories"
11 USER_STORIES_VECTOR_INDEX_NAME="vector_index_user_story"
12
13 #USER_STORIES_COLLECTION_NAME="user_stories"
14 #USER_STORIES_VECTOR_INDEX_NAME="vector_index"
15
16 # Mistral AI Configuration (for embeddings)
17 MISTRAL_API_KEY=frpypa50EIAjplVj2y659rctb8h2A
18 MISTRAL_EMBEDDING_MODEL="mistral-embed"
```

PS C:\GenAIForQA\Phase2\Week03\rag-mongo-demo-backend\rag-mongo-demo-backend> node src/scripts/embeddings/create-embeddings-batch-mistral.js
Using Mistral AI Embedding API for Cost-Effective Processing!
Configuration:
- Batch Size: 50 testcases per API call
- Concurrent Batch calls: 3
- MongoDB Batch Size: 100
- Delay Between Batch Groups: 1000ms
- API: Mistral AI (mistral-embed)
- Embedding Dimensions: 1024
- Database: db_testcases
- Collection: TestCases
- Total Batches: 128
- Batch Groups: 43
- Estimated Time: 2.9 minutes
[Batch 1/128] Processing 50 testcases with Mistral AI...
[Batch 2/128] Processing 50 testcases with Mistral AI...
[Batch 3/128] Processing 50 testcases with Mistral AI...
[Batch 1/128] Success! Cost: \$0.000768 | Tokens: 7682
[Batch 1/128] Success! Cost: \$0.000736 | Tokens: 7355
[Batch 2/128] Success! Cost: \$0.000818 | Tokens: 8175

```
.env
1 MONGODB_URI=mongodb+srv://suryapsgtech_db_user:ChummsZ4001@cluster0.owtcttv.r
2 #MONGODB_URI=mongodb+srv://sumitha:TestLeaf321@cluster0.c514wjg.mongodb.net/
3 DB_NAME="db_testcases"
4 #DB_NAME="RagTest_cases"
5
6 COLLECTION_NAME="TestCases"
7 VECTOR_INDEX_NAME="vector_index_test_cases"
8 #BM25_INDEX_NAME="bm25_search"
9
10 USER_STORIES_COLLECTION_NAME="user_stories"
11 USER_STORIES_VECTOR_INDEX_NAME="vector_index_user_story"
12
13 #USER_STORIES_COLLECTION_NAME="user_stories"
14 #USER_STORIES_VECTOR_INDEX_NAME="vector_index"
15
16 # Mistral AI Configuration (for embeddings)
17 MISTRAL_API_KEY=wfrpypas0EIAjplVj2wyG5yrcTwh2A
18 MISTRAL_EMBEDDING_MODEL="mistral-embed"

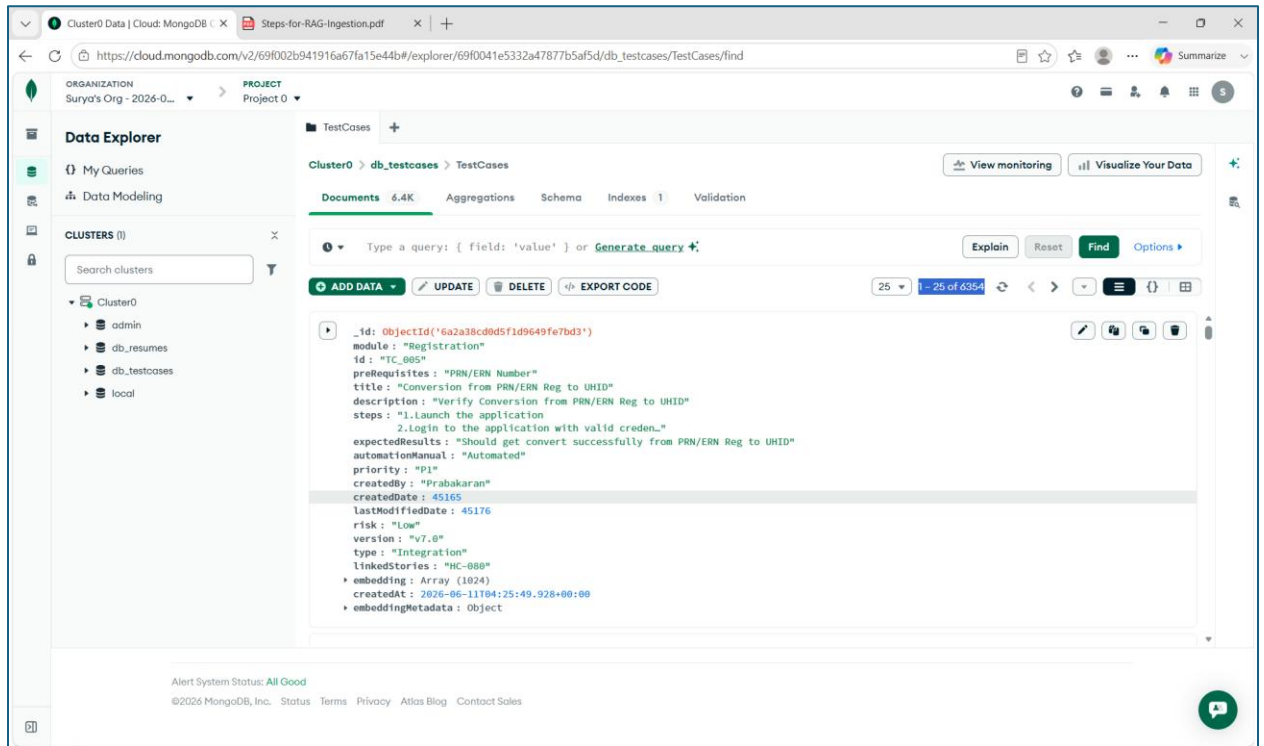
PS C:\GenAIForQA\Phase2\Week03\rag-mongo-demo-backend\rag-mongo-demo-backend> node src/scripts/embeddings/create-embeddings-batch-mistral.js
Waiting 1000ms before next batch group...
[Batch 127/128] Processing 50 testcases with Mistral AI...
[Batch 128/128] Processing 4 testcases with Mistral AI...
[Batch 128/128] Success! Cost: $0.000074 | Tokens: 744
[Batch 127/128] Success! Cost: $0.001032 | Tokens: 10317
Progress: 6354/6354 (100.0%) | Rate: 11.5/sec | ETA: 0s | Cost: $0.117741

MISTRAL AI BATCH PROCESSING COMPLETE!
Final Statistics:
  Total Times: 9m 13s
  Processing Rate: 11.5 testcases/second
  Total Test Cases: 6354
  Successfully Processed: 6354
  Failed: 0
  Success Rate: 100.0%
  Total Cost: $0.117741
  Total Tokens: 1,177,411
  Average Cost per Test Case: $0.0001853
  Average Tokens per Test Case: 185
  Cost Savings: >90% cheaper than OpenAI embeddings
```

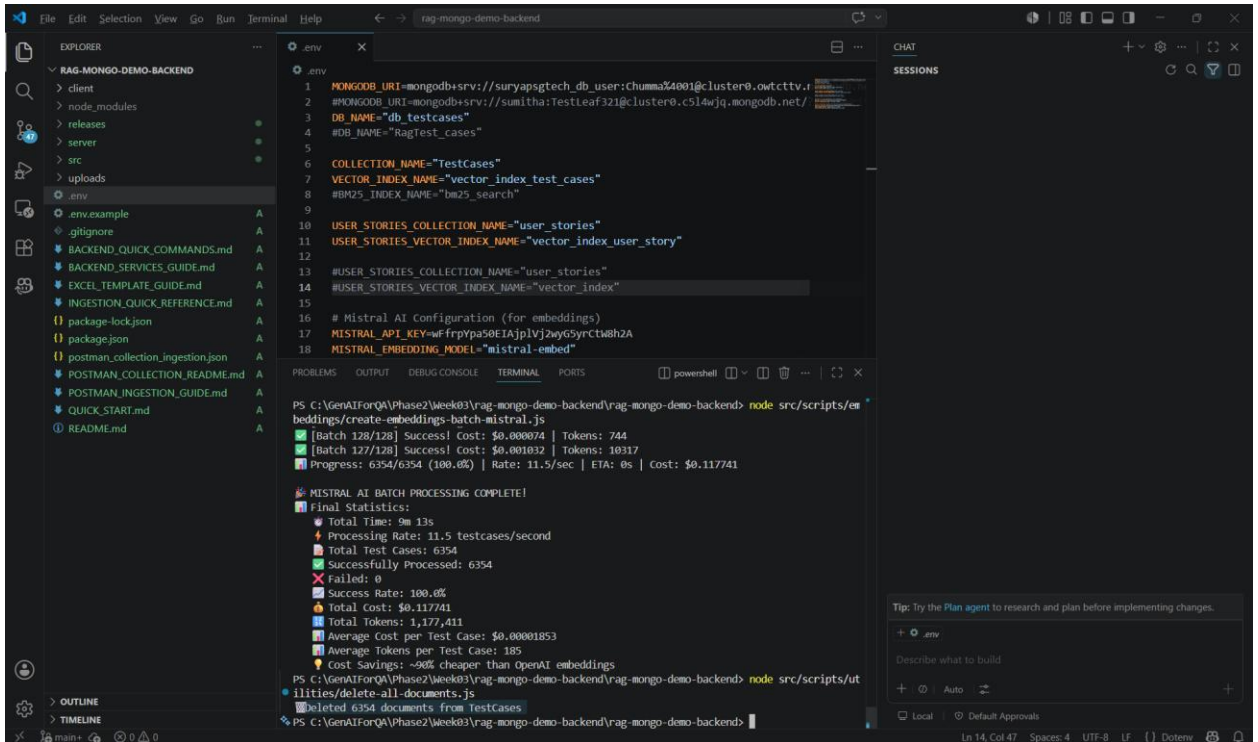
MongoDB Verification - Testcases upload in MongoDB

Database	Collection	Index Name	Status	Queryable	Type	Index Fields	Documents (Estimated)	Size
db_testcases	TestCases	vector_index_test_cases	READY	✓	vectorSearch	"embedding", "id", "module", "title", "description"	6,354 (100%) indexed of 6,354	18.17MB

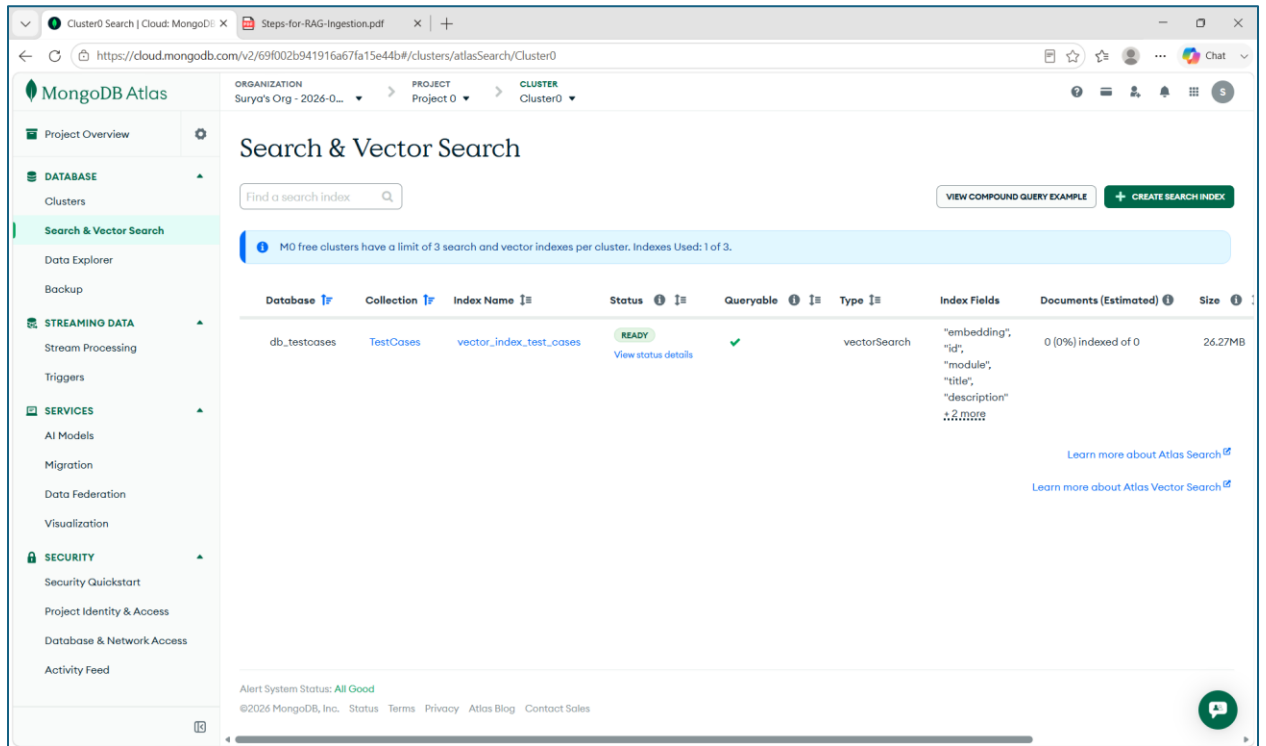
All 6354 testcases data inserted into MongoDB



Deleting all the data to proceed for API verification
node src/scripts/utilities/delete-all-documents.js

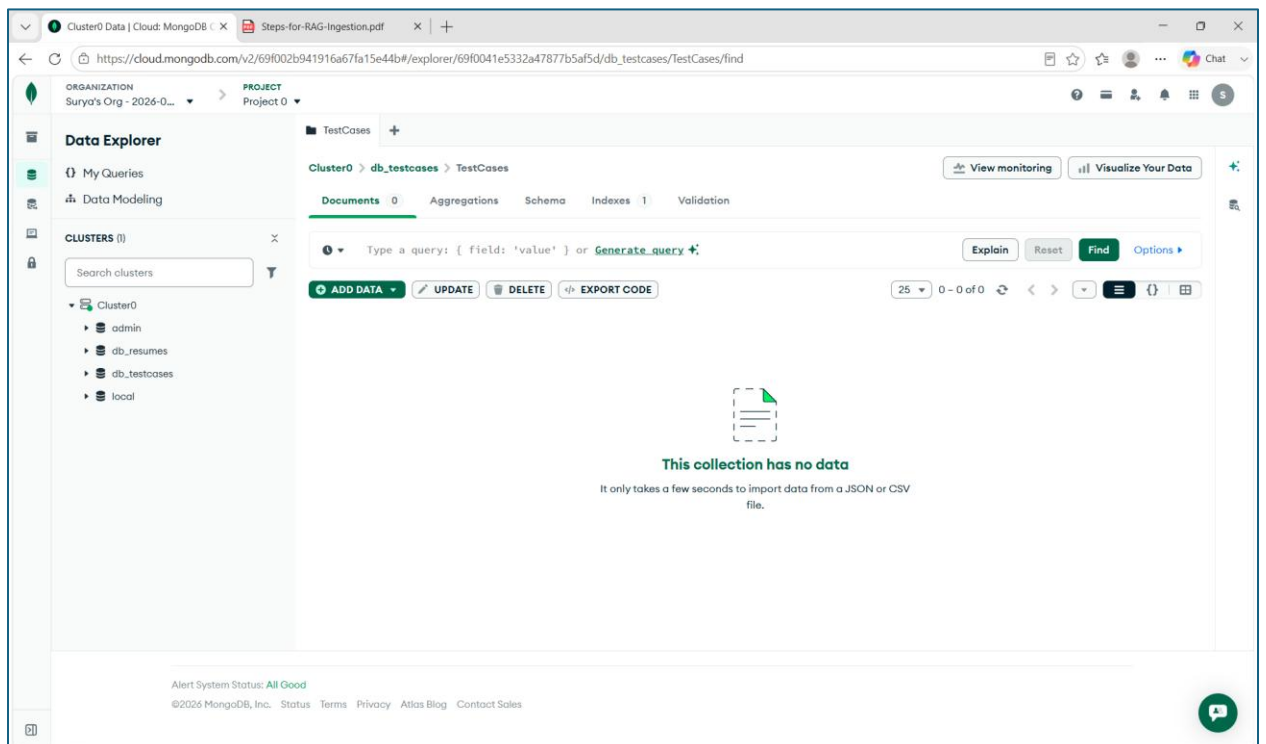


No Data in MongoDB



The screenshot shows the MongoDB Atlas 'Search & Vector Search' interface. The left sidebar contains navigation links for Project Overview, DATABASE (Clusters), Search & Vector Search (selected), Data Explorer, Backup, STREAMING DATA (Stream Processing, Triggers), SERVICES (AI Models, Migration, Data Federation, Visualization), and SECURITY (Security Quickstart, Project Identity & Access, Database & Network Access, Activity Feed). The main content area has a search bar and a table of search indexes. A blue banner at the top states: 'M0 free clusters have a limit of 3 search and vector indexes per cluster. Indexes Used: 1 of 3.' The table lists one index: 'vector_index_test_cases' in the 'TestCases' collection of the 'db_testcases' database. The index is 'READY', 'Queryable', and of type 'vectorSearch'. It has fields: 'embedding', 'id', 'module', 'title', and 'description'. The documents are estimated at 0 (0% indexed of 0) and the size is 26.27MB. Links for 'Learn more about Atlas Search' and 'Learn more about Atlas Vector Search' are provided. The footer shows 'Alert System Status: All Good' and copyright information for MongoDB, Inc.

Database	Collection	Index Name	Status	Queryable	Type	Index Fields	Documents (Estimated)	Size
db_testcases	TestCases	vector_index_test_cases	READY	✓	vectorSearch	"embedding", "id", "module", "title", "description"	0 (0%) indexed of 0	26.27MB



The screenshot shows the MongoDB Atlas 'Data Explorer' interface. The left sidebar has links for My Queries, Data Modeling, and CLUSTERS (1). The main content area shows the 'TestCases' collection in the 'db_testcases' database. It has tabs for Documents (selected), Aggregations, Schema, Indexes (1), and Validation. A query bar shows a placeholder: 'Type a query: { field: 'value' } or Generate query'. Below the query bar are buttons for ADD DATA, UPDATE, DELETE, and EXPORT CODE. The main area displays a message: 'This collection has no data. It only takes a few seconds to import data from a JSON or CSV file.' The footer shows 'Alert System Status: All Good' and copyright information for MongoDB, Inc.

API Verification

Step1: Health Check

The screenshot shows the Postman interface for a GET request to `GET 1. Health Check` at the endpoint `[[baseUri]] /api/health`. The request is successful, returning a 200 OK status with a response time of 49 ms and a body size of 312 B. The response body is a JSON object:

```
{
  "status": "OK",
  "message": "Server is running"
}
```

The left sidebar shows a list of collections, including 'RAG MongoDB - Data Ingestion' and 'restful-booker-api'. The 'What's next?' button is visible under the 'ENVIRONMENTS' section.

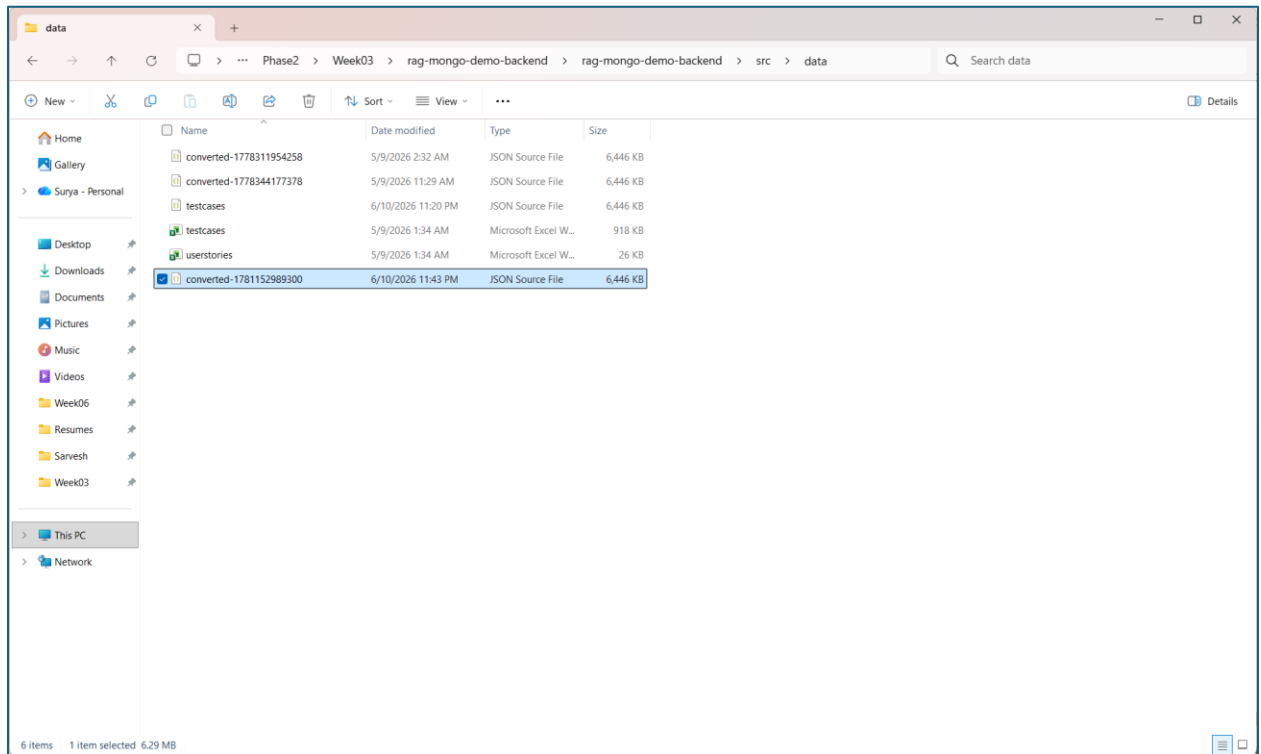
Step2: Upload Excel

The screenshot shows the Postman interface for a POST request to `POST 2. Upload Excel - Test Cases` at the endpoint `[[baseUri]] /api/upload-excel`. The request is successful, returning a 200 OK status with a response time of 1.07 s and a body size of 825 B. The response body is a JSON object:

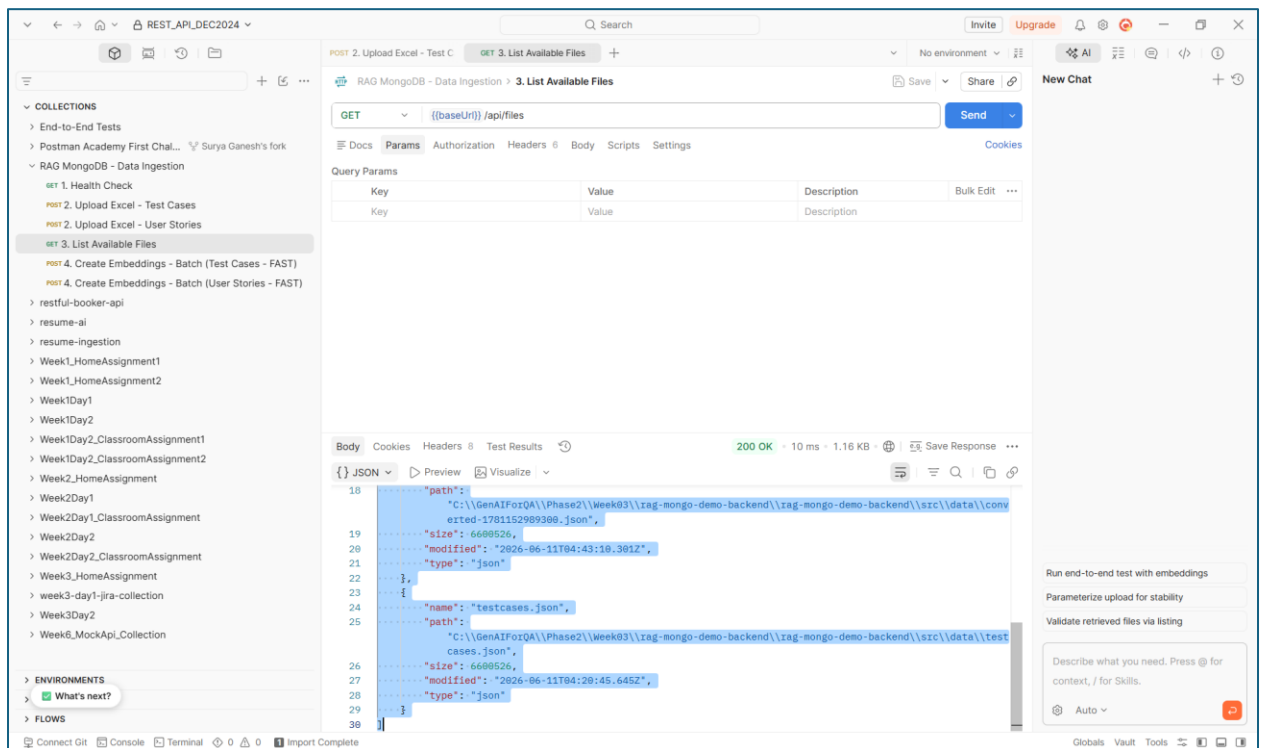
```
{
  "success": true,
  "message": "Test cases file converted successfully",
  "outputFile": "converted-1781152989398.json",
  "output": "⚠️ Sheet 'Testcases' not found. Using first sheet: 'testcases'\n✅ Converted 6354 rows from 'testcases' into C:/GenAIForQA/Phase2/week83/rag-mongo-demo-backend/rag-mongo-demo-backend/src/data/converted-1781152989398.json\n"
}
```

The left sidebar shows a list of collections, including 'RAG MongoDB - Data Ingestion' and 'restful-booker-api'. The 'What's next?' button is visible under the 'ENVIRONMENTS' section.

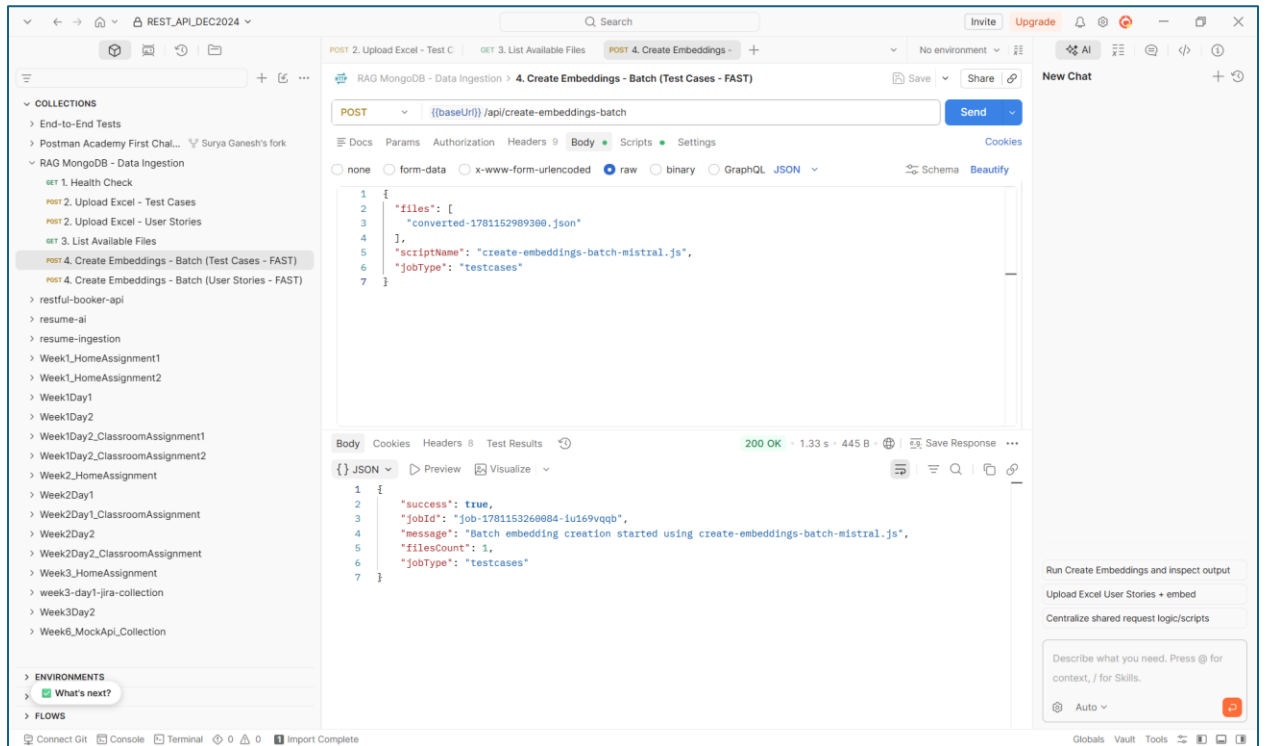
Excel to JSON conversion and the converted json is in the path – src\data



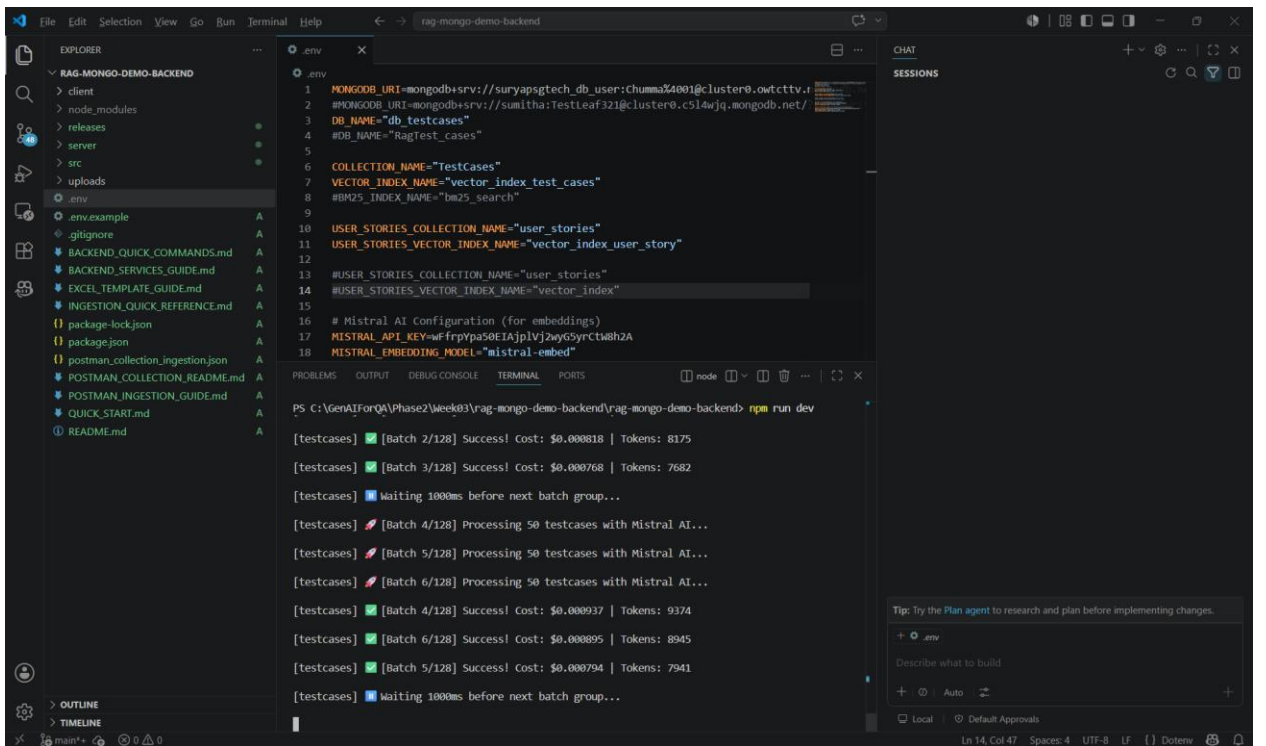
Step3: List Available Files



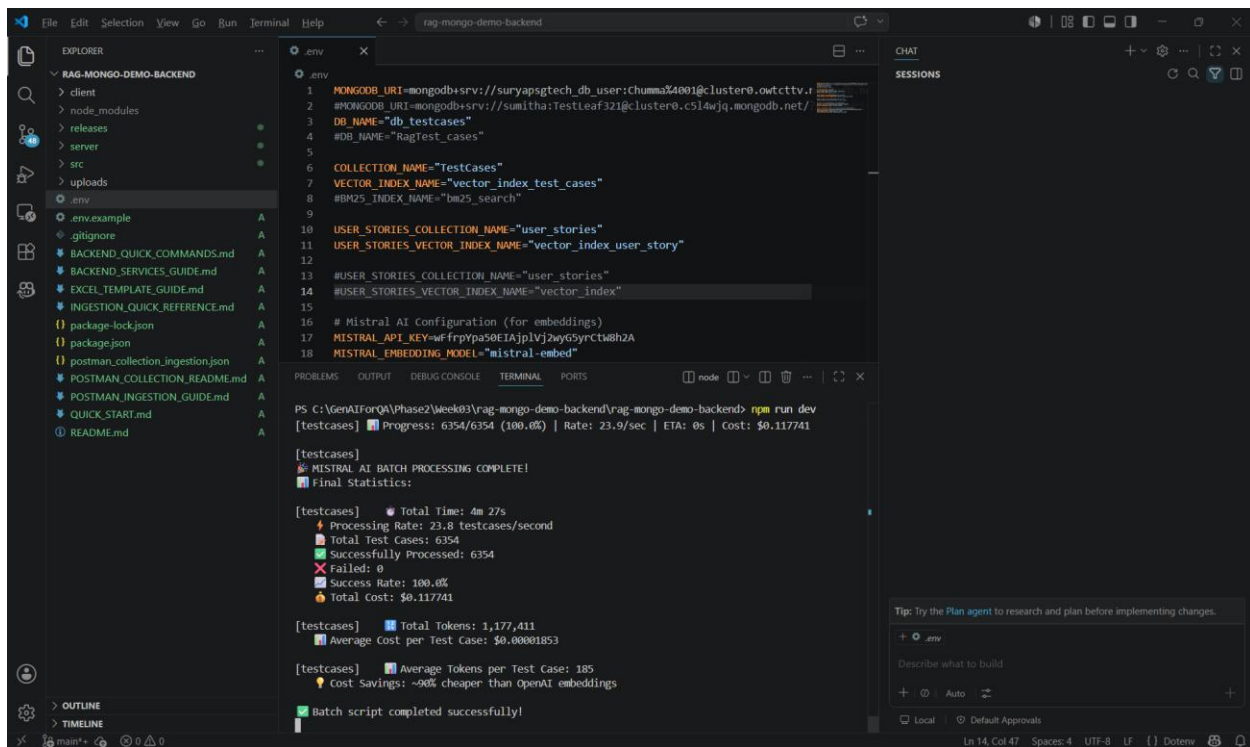
Step4: Create Embeddings for batch processing



Batch processing starts

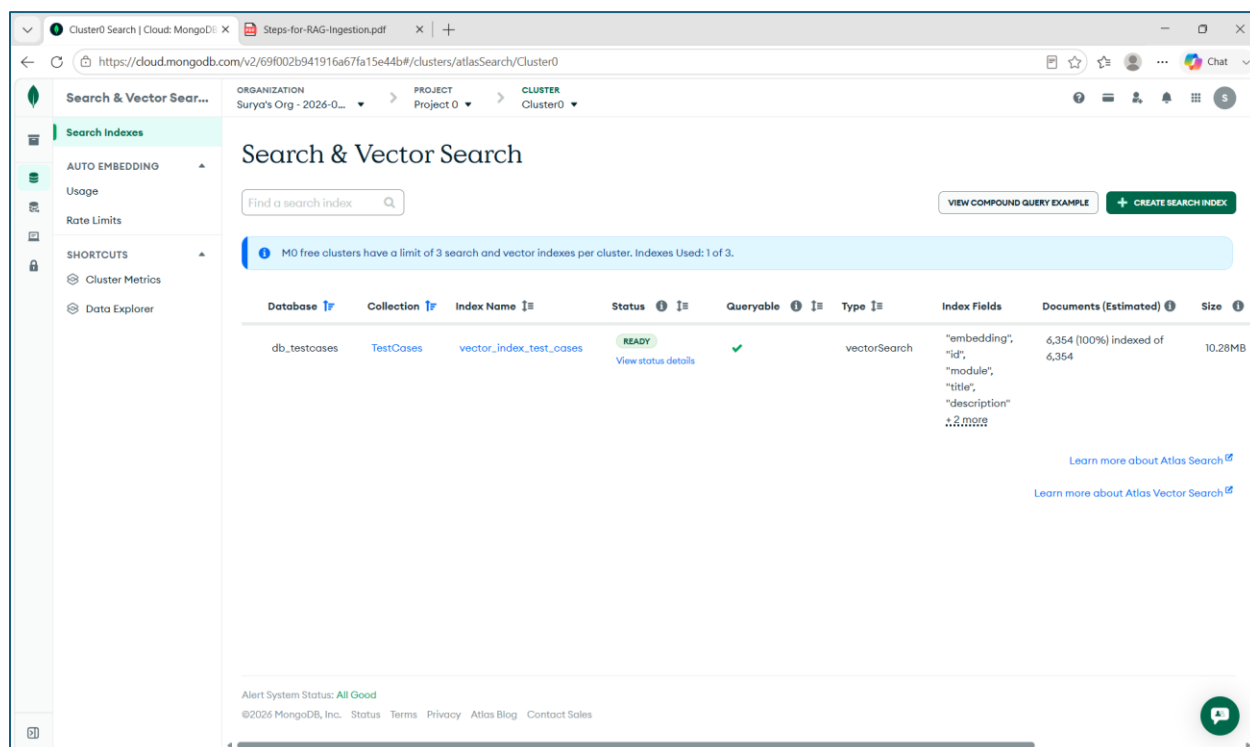


Successful completion of batch processing from API



```
1 MONGODB_URI=mongodb+srv://suryapsrtech_db_user:chumma54001@cluster0.owtcttv.mongodb.net/
2 #MONGODB_URI=mongodb+srv://sumitha:TestLeaf321@cluster0.c514wjq.mongodb.net/
3 DB_NAME="db_testcases"
4 #DB_NAME="RagTest_cases"
5
6 COLLECTION_NAME="TestCases"
7 VECTOR_INDEX_NAME="vector_index_test_cases"
8 #BM25_INDEX_NAME="bm25_search"
9
10 USER_STORIES_COLLECTION_NAME="user_stories"
11 USER_STORIES_VECTOR_INDEX_NAME="vector_index_user_story"
12
13 #USER_STORIES_COLLECTION_NAME="user_stories"
14 #USER_STORIES_VECTOR_INDEX_NAME="vector_index"
15
16 # Mistral AI Configuration (for embeddings)
17 MISTRAL_API_KEY=wfrpypa50EIAjplVj2wyG5yrCttW8h2A
18 MISTRAL_EMBEDDING_MODEL="mistral-embed"
19
20 PS C:\GenAIForQA\Phase2\Week03\rag-mongo-demo-backend\rag-mongo-demo-backend> npm run dev
[testcases] Progress: 6354/6354 (100.0%) | Rate: 23.9/sec | ETA: 0s | Cost: $0.117741
[testcases]
[testcases] MISTRAL AI BATCH PROCESSING COMPLETE!
[testcases] Final Statistics:
[testcases] Total time: 4m 27s
[testcases] Processing Rate: 23.8 testcases/second
[testcases] Total Test Cases: 6354
[testcases] Successfully Processed: 6354
[testcases] Failed: 0
[testcases] Success Rate: 100.0%
[testcases] Total Cost: $0.117741
[testcases] Total Tokens: 1,177,411
[testcases] Average Cost per Test Case: $0.0001853
[testcases] Average Tokens per Test Case: 185
[testcases] Cost Savings: ~90% cheaper than OpenAI embeddings
[testcases] Batch script completed successfully!
```

MongoDB Verification



Database	Collection	Index Name	Status	Queryable	Type	Index Fields	Documents (Estimated)	Size
db_testcases	TestCases	vector_index_test_cases	READY	✓	vectorSearch	"embedding", "id", "module", "title", "description"	6,354 (100%) indexed of 6,354	10.28MB

Cluster0 Data | Cloud: MongoDB

Steps-for-RAG-Ingestion.pdf

https://cloud.mongodb.com/v2/69f002b941916a67fa15e44b#/explorer/69f0041e5332a47877b5af5d/db_testcases/TestCases/find

ORGANIZATION
Surya's Org - 2026-0...

PROJECT
Project 0

TestCases

+

View monitoring

Visualize Your Data

Data Explorer

My Queries

Data Modeling

CLUSTERS (1)

Search clusters

Cluster0

- admin
- db_resumes
- db_testcases
- local

Cluster0 > db_testcases > TestCases

Documents 6.4K Aggregations Schema Indexes 1 Validation

Type a query: { field: 'value' } or [Generate query](#)

ADD DATA

UPDATE

DELETE

EXPORT CODE

25 1 - 25 of 6354

```
{
  "_id": ObjectId("6a2a3defb574ff4637a2766f"),
  "module": "Registration",
  "id": "TC_005",
  "prerequisites": "PRN/ERN Number",
  "title": "Conversion from PRN/ERN Reg to UHID",
  "description": "Verify Conversion from PRN/ERN Reg to UHID",
  "steps": "1.Launch the application\n2.Login to the application with valid creden..",
  "expectedResults": "Should get convert successfully from PRN/ERN Reg to UHID",
  "automationManual": "Automated",
  "priority": "P1",
  "createdBy": "Prabakaran",
  "createdAt": 45165,
  "lastModifiedDate": 45176,
  "linkedStories": "HC-080",
  "risk": "Low",
  "version": "v7.0",
  "type": "Integration",
  "embedding": Array (1024),
  "createdAt": 2026-06-11T04:47:43.426+00:00,
  "embeddingMetadata": Object
}
```

Alert System Status: All Good

©2026 MongoDB, Inc. Status Terms Privacy Atlas Blog Contact Sales

Chat

RAG Ingestion – Backend for Userstories

- a. Database ->Collection created in MongoDB
- b. Vector Index for User stories also created in MongoDB

The screenshot displays the MongoDB Atlas Search & Vector Search interface. The left sidebar shows navigation options: Search & Vector Search, Search Indexes, AUTO EMBEDDING, Usage, Rate Limits, SHORTCUTS, Cluster Metrics, and Data Explorer. The main content area is titled 'Search & Vector Search' and includes a search bar, a 'VIEW COMPOUND QUERY EXAMPLE' button, and a '+ CREATE SEARCH INDEX' button. A blue banner indicates that M0 free clusters have a limit of 3 search and vector indexes per cluster, with 2 of 3 indexes used. Below this, a table lists the search indexes:

Database	Collection	Index Name	Status	Queryable	Type	Index Fields	Documents (Estimated)	Size
db_testcases	TestCases	vector_index_test_cases	READY	✓	vectorSearch	"embedding", "id", "module", "title", "description",...2 more	6,354 (100%) indexed of 6,354	26.26
db_testcases	UserStories	vector_index_userstories	READY	✓	vectorSearch	"embedding", "projectName", "statusCategory", "priority", "risk",...1 more	190 (100%) indexed of 190	794.12

At the bottom of the interface, there is a footer with the text 'Alert System Status: All Good', '©2026 MongoDB, Inc.', and links for 'Status', 'Terms', 'Privacy', 'Atlas Blog', and 'Contact Sales'. A chat icon is also present in the bottom right corner.

Step1: Run the command node src/scripts/data-conversion/excel-to-userstories.js

```
src > scripts > data-conversion > JS excel-to-userstories.js > ...
1 import xlsx from "xlsx";
2 import fs from "fs";
3
4 // CONFIGURATION
5 const excelFile = "src/data/userstories.xlsx";
6 const sheetName = "stories"; // Update this based on your Excel sheet name
7 const outputFile = "src/data/stories.json";
8
9 // Map Excel column headers to JSON keys (based on common user story fields)
10 const columnMap = {
11   // Core user story fields - Excel already uses lowercase/camelcase
12   "storyId": "key",
13   "Story Key": "key",
14   "Story ID": "key",
15   "ID": "key",
16   "Key": "key",
17   "Issue Key": "key",
18
19   // ... other mappings ...
20 };
21
22 // ... rest of the script ...
```

node src/scripts/data-conversion/excel-to-userstories.js

SUCCESS! Converted user stories from Excel to JSON
Input: src/data/userstories.xlsx (sheet: "stories")
Output: src/data/stories.json
Total user stories: 190

Sample converted user story:
Key: HC-257
Summary: Consultant Dashboard My Inpatients
Status: To Do
Priority: P1
Story Points: Not set
Project: Health Care

Next steps:
1. Review the generated src/data/stories.json file
2. Run: node src/scripts/create-userstories-embeddings-batch.js
3. Create vector index in MongoDB Atlas for user stories search

Stories.json file is created in \src\data

```
src > data > stories.json > ...
1 {
2   "key": "HC-257",
3   "summary": "Consultant Dashboard My Inpatients",
4   "description": "As a user, I want to consultant dashboard my inpatients",
5   "status": {
6     "name": "To Do",
7     "category": "To Do"
8   },
9   "priority": {
10    "name": "P1",
11    "id": "3"
12  },
13  "assignee": null,
14  "reporter": null,
15  "created": "2022-01-06T00:00:00.000Z",
16  "updated": "2020-06-11T05:52:31.863Z",
17  "components": []
18 }
```

node src/scripts/data-conversion/excel-to-userstories.js

SUCCESS! Converted user stories from Excel to JSON
Input: src/data/userstories.xlsx (sheet: "stories")
Output: src/data/stories.json
Total user stories: 190

Sample converted user story:
Key: HC-257
Summary: Consultant Dashboard My Inpatients
Status: To Do
Priority: P1
Story Points: Not set
Project: Health Care

Next steps:
1. Review the generated src/data/stories.json file
2. Run: node src/scripts/create-userstories-embeddings-batch.js
3. Create vector index in MongoDB Atlas for user stories search

Step2: Creation of embeddings - node src/scripts/embeddings/create-userstories-embeddings-batch-mistral.js

```
src > scripts > embeddings > JS create-userstories-embeddings-batch-mistral.js > ...
1 import { MongoClient } from "mongodb";
2 import dns from "dns";
3 import dotenv from "dotenv";
4 import fs from "fs";
5 import path from "path";
6 import pLimit from "p-limit";
7 import { generateBatchEmbeddings } from "../utilities/mistralEmbedding.js";
8
9 dotenv.config();
10
11 // Fix DNS resolution issue on macOS by using Google's DNS servers
12 dns.setServers(["8.8.8.8", "8.8.4.4"]);
13
14 // Configure MongoDB client with SSL options
15 const client = new MongoClient(process.env.MONGODB_URI, {
16   ssl: true,
17   tlsAllowInvalidCertificates: true,
18   tlsAllowInvalidHostnames: true,
19   serverSelectionTimeoutMS: 30000,
20   connectTimeoutMS: 30000,
21   socketTimeoutMS: 30000,
22 });
```

PS C:\GenAIForQA\Phase2\Week03\rag-mongo-demo-backend\rag-mongo-demo-backend>

MISTRAL AI BATCH PROCESSING COMPLETE!

Final Statistics:

- Total Time: 8s
- Processing Rate: 24.5 user stories/second
- Total User Stories: 190
- Successfully Processed: 190
- Failed: 0
- Success Rate: 100.0%
- Total Cost: \$0.003968
- Total Tokens: 39,678
- Average Cost per User Story: \$0.00001615
- Average Tokens per User Story: 161
- Cost Savings: ~90% cheaper than OpenAI embeddings

Tip: Try the Plan agent to research and plan before implementing changes.

+ create-userstories-embeddings-batch-mistral.js

Describe what to build

+ Auto

Local Default Approvals

Ln 1, Col 1 Spaces: 2 UTF-8 LF JavaScript

Successfully injected 190 user stories

MongoDB Verification (Already 190 user stories were available, so now 380)

The screenshot shows the 'Search & Vector Search' page in the MongoDB Atlas console. The left sidebar contains navigation options: Search Indexes, AUTO EMBEDDING, Usage, Rate Limits, SHORTCUTS, Cluster Metrics, and Data Explorer. The main content area has a search bar and a table of search indexes. A blue banner at the top states: 'M0 free clusters have a limit of 3 search and vector indexes per cluster. Indexes Used: 2 of 3.'

Database	Collection	Index Name	Status	Queryable	Type	Index Fields	Documents (Estimated)	Size
db_testcases	TestCases	vector_index_test_cases	READY	✓	vectorSearch	"embedding", "id", "module", "title", "description", "risk"	6,354 (100%) indexed of 6,354	26.26 MB
db_testcases	UserStories	vector_index_userstories	READY	✓	vectorSearch	"embedding", "projectName", "statusCategory", "priority", "risk"	380 (100%) indexed of 380	794.12 MB

Alert System Status: All Good
©2026 MongoDB, Inc. Status Terms Privacy Atlas Blog Contact Sales

The screenshot shows the 'Data Explorer' page in the MongoDB Atlas console. The left sidebar contains navigation options: My Queries, Data Modeling, CLUSTERS (1), and a search bar. The main content area shows the 'UserStories' collection with 380 documents. A document is selected and its details are displayed in a JSON format.

```
{
  "_id": ObjectId('6a2a497f9f8da0cf51f0a0b9'),
  "key": "HC-257",
  "summary": "Consultant Dashboard My Inpatients",
  "description": "As a user, I want to consultant dashboard my inpatients so that the re.",
  "status": Object,
  "priority": Object,
  "assignee": null,
  "reporter": null,
  "created": "2022-01-06T00:00:00.000Z",
  "updated": "2026-06-11T05:34:28.338Z",
  "components": Array (empty),
  "labels": Array (empty),
  "fixVersions": Array (empty),
  "storyPoints": null,
  "project": "Health Care",
  "epic": "Emergency Management",
  "acceptanceCriteria": "Given a user with valid access, when they open the dashboard, then reL",
  "businessValue": "",
  "risk": "Critical",
  "dependencies": "",
  "notes": "",
  "sprint": null,
  "team": null
}
```

Alert System Status: All Good
©2026 MongoDB, Inc. Status Terms Privacy Atlas Blog Contact Sales

API Verification

Available data in MongoDB - 380

The screenshot displays the MongoDB Atlas Search & Vector Search interface. The left sidebar shows navigation options: Search & Vector Search, Search Indexes, AUTO EMBEDDING, Usage, Rate Limits, SHORTCUTS, Cluster Metrics, and Data Explorer. The main panel is titled 'Search & Vector Search' and includes a search bar and a button to 'CREATE SEARCH INDEX'. A notification states: 'M0 free clusters have a limit of 3 search and vector indexes per cluster. Indexes Used: 2 of 3.' Below this, a table lists the search indexes:

Database	Collection	Index Name	Status	Queryable	Type	Index Fields	Documents (Estimated)	Size
db_testcases	TestCases	vector_index_test_cases	READY	✓	vectorSearch	"embedding", "id", "module", "title", "description", "risk", "priority"	0 (100%) indexed of 0	26.2K
db_testcases	UserStories	vector_index_userstories	READY	✓	vectorSearch	"embedding", "projectName", "statusCategory", "priority", "risk", "priority"	380 (100%) indexed of 380	1.52M

At the bottom, there is a status bar indicating 'Alert System Status: All Good' and a chat icon.

Step1: Health Check

The screenshot shows the REST Client interface with a collection of API endpoints. The selected endpoint is 'GET 1. Health Check' with the URL 'GET {{baseUrl}} /api/health'. The request is a GET method. The response is a 200 OK status with a 31 ms response time and 312 B of data. The response body is shown in JSON format:

```
{
  "status": "OK",
  "message": "Server is running"
}
```

The interface also includes a sidebar with a list of collections and environments, and a chat window on the right.

Step2: Upload Userstories

The screenshot shows the Postman interface with a REST client request configured for a POST method to the endpoint `/(baseUri)/api/upload-excel`. The request body is set to `form-data` and contains three fields: `file` (value: `userstories.xlsx`), `dataType` (value: `userstories`), and `sheetName` (value: `stories`). The response is a 200 OK status with a JSON body indicating success.

```
{
  "success": true,
  "message": "User stories file converted successfully",
  "outputFile": "stories-1781158087727.json",
  "output": "Converting 190 rows from sheet 'stories'... Converted 190 user stories from 'stories' into C:/GenAIForQA/Phase2/Week03/rag-mongo-demo-backend/src/data/stories-1781158087727.json. Filtered out: 0 empty rows"
}
```

Converted USerstories json file is available in `\src\data`

The screenshot shows a Windows File Explorer window displaying the contents of the `\src\data` directory. The directory contains several JSON source files and two Excel files. The file `stories-1781158087727` is selected.

Name	Date modified	Type	Size
converted-1778311954258	5/9/2026 2:32 AM	JSON Source File	6,446 KB
converted-1778344177378	5/9/2026 11:29 AM	JSON Source File	6,446 KB
converted-1781152989300	6/10/2026 11:43 PM	JSON Source File	6,446 KB
stories	6/11/2026 12:52 AM	JSON Source File	199 KB
stories-1781158087727	6/11/2026 1:08 AM	JSON Source File	199 KB
testcases	6/11/2026 12:50 AM	JSON Source File	6,446 KB
testcases	5/9/2026 1:34 AM	Microsoft Excel W...	918 KB
userstories	5/9/2026 1:34 AM	Microsoft Excel W...	26 KB

Step3: List available files

The screenshot shows the Postman interface with a GET request to `/(baseUrl)/api/files`. The response is a 200 OK status with a 7 ms response time and 1.58 KB of data. The response body is a JSON array of file objects. The first object is:

```
{  "path": "C:\\GenAIForQ4\\Phase2\\Week83\\rag-mongo-demo-backend\\rag-mongo-demo-backend\\src\\data\\converted-178115299398.json",  "size": 6689526,  "modified": "2026-06-11T04:43:10.301Z",  "type": "json"}
```

The second object is:

```
{  "name": "stories-178115887727.json",  "path": "C:\\GenAIForQ4\\Phase2\\Week83\\rag-mongo-demo-backend\\rag-mongo-demo-backend\\src\\data\\stories-178115887727.json",  "size": 203946,  "modified": "2026-06-11T06:08:08.112Z",  "type": "json"}
```

The interface also shows a sidebar with a list of collections and environments, and a right-hand panel with a 'New Chat' button and a 'Describe what you need' input field.

Step4: Create embeddings

The screenshot shows the Postman interface with a POST request to `/(baseUrl)/api/create-embeddings-batch`. The response is a 200 OK status with a 1.43 s response time and 459 B of data. The response body is a JSON object:

```
{  "success": true,  "jobId": "job-1781158264387-tum2uk59",  "message": "Batch embedding creation started using create-userstories-embeddings-batch-mistral.js",  "filesCount": 1,  "jobType": "userstories"}
```

The interface also shows a sidebar with a list of collections and environments, and a right-hand panel with a 'New Chat' button and a 'Describe what you need' input field.

Successful completion of batch processing

The screenshot shows a VS Code editor interface with the following components:

- Explorer Panel:** Displays the project structure for 'rag-mongo-demo-backend'. The 'scripts' directory is expanded, showing 'delete-all-documents.js' as the active file.
- Main Editor:** Contains the JavaScript code for 'delete-all-documents.js'. The code uses the 'mongodb' client to connect to a database and delete all documents from a collection. It includes error handling and logging.
- Terminal Panel:** Shows the command 'npm run dev' executed in the terminal. The output indicates that the batch processing is complete and provides the following statistics:
 - Final Statistics:**
 - Total Time: 8s
 - Processing Rates: 23.7 user stories/second
 - Total User Stories: 190
 - Successfully Processed: 190
 - Failed: 0
 - Success Rate: 100.0%
 - Total Cost: \$0.003068
 - Token Usage:**
 - Total Tokens: 30,678
 - Average Cost per User Story: \$0.00001615
 - Average Tokens per User Story: 161
 - Cost Savings: ~90% cheaper than OpenAI embeddings
 - Batch script completed successfully!**
- Chat Panel:** Displays a tip from the Plan agent: 'Tip: Try the Plan agent to research and plan before implementing changes.' It also shows a search for 'delete-all-documents.js'.

MongoDB verification

The screenshot shows the 'Search & Vector Search' page in the MongoDB Atlas console. The left sidebar contains navigation links for 'Search & Vector Search', 'AUTO EMBEDDING', 'Usage', 'Rate Limits', 'SHORTCUTS', 'Cluster Metrics', and 'Data Explorer'. The main content area has a search bar and a table of search indexes. A blue banner at the top states: 'M0 free clusters have a limit of 3 search and vector indexes per cluster. Indexes Used: 2 of 3.'

Database	Collection	Index Name	Status	Queryable	Type	Index Fields	Documents (Estimated)	Size
db_testcases	TestCases	vector_index_test_cases	READY	✓	vectorSearch	"embedding", "id", "module", "title", "description", "priority", "risk", "statusCategory", "summary"	0 (0%) indexed of 0	317B
db_testcases	UserStories	vector_index_userstories	READY	✓	vectorSearch	"embedding", "projectName", "statusCategory", "priority", "risk", "summary"	570 (100%) indexed of 570	1.52M

Alert System Status: All Good
©2026 MongoDB, Inc. Status Terms Privacy Atlas Blog Contact Sales

The screenshot shows the 'Data Explorer' page in the MongoDB Atlas console. The left sidebar contains navigation links for 'Data Explorer', 'My Queries', 'Data Modeling', and 'CLUSTERS (1)'. The main content area shows the 'UserStories' collection with 570 documents. A document is selected and its details are displayed in a JSON format.

```
{
  "_id": ObjectId('6a2a497f9f8da0cf51f0a0b9'),
  "key": "HC-257",
  "summary": "Consultant Dashboard My Inpatients",
  "description": "As a user, I want to consultant dashboard my inpatients so that the re.",
  "status": Object,
  "priority": Object,
  "assignee": null,
  "reporter": null,
  "created": "2022-01-06T00:00:00.000Z",
  "updated": "2026-06-11T05:34:28.338Z",
  "components": Array (empty),
  "Labels": Array (empty),
  "fixVersions": Array (empty),
  "storyPoints": null,
  "project": "Health Care",
  "epic": "Emergency Management",
  "acceptanceCriteria": "Given a user with valid access, when they open the dashboard, then reL",
  "businessValue": "",
  "risk": "Critical",
  "dependencies": "",
  "notes": "",
  "sprint": null,
  "team": null
}
```

Alert System Status: All Good
©2026 MongoDB, Inc. Status Terms Privacy Atlas Blog Contact Sales